

CPU Thermal Prediction Initial Project Guide

2026-06-27

1. Project overview

This project is about building a machine learning pipeline that can **predict imminent thermal stress / thermal distress on an Apple Silicon Mac before the event fully manifests**. The central idea is not merely to detect that the machine is already hot, but to learn from telemetry patterns so that the system can identify when a problematic thermal state is likely to occur in the near future.

At a high level, the work combines **systems telemetry collection**, **time-series data engineering**, **label construction**, **exploratory data analysis**, and the upcoming **supervised classification** stage. The end product is intended to be a predictive tabular ML workflow in which each row represents a timestamped telemetry snapshot, enriched with short-term temporal context through engineered features, and mapped to a forward-looking binary target indicating whether thermal distress is approaching within a chosen prediction horizon.

In practical terms, the project asks a clear statistical machine learning question:

Given present and recent telemetry measurements from an Apple Silicon system, can a model predict whether the CPU temperature will cross a defined thermal threshold within the next fixed time horizon?

That question is important because it reframes raw monitoring into **early-warning prediction**. If successful, such a system could support intelligent workload adaptation, system diagnostics, or future thermal-management policies.

2. Core objective

The operational objective of the project is to construct a dataset and pipeline that support the following supervised learning task:

- **Input:** current and recent telemetry signals such as temperature, power, activity, frequency, load, and memory indicators.
- **Output:** a binary label indicating whether a thermal event is expected soon.
- **Prediction type:** early-warning classification.
- **Data form:** session-based time-series converted into a feature-rich tabular dataset.

The target used in the current implementation is not just “machine is hot now.” Instead, the label is built using a **look-ahead window**. A row becomes positive when the target event will happen within the next chosen time horizon, which in the notebook currently appears to be 1 minute, with 5-second telemetry sampling, implying 12 future steps in the prediction window.

This design is very important conceptually. It means the model is trained to anticipate thermal escalation, not merely classify already-observed overheating.

3. Why this project matters

From a machine learning perspective, this is an interesting problem because it sits at the intersection of:

- systems telemetry,
- weakly rare-event prediction,
- time-series-to-tabular feature engineering,
- class imbalance handling,
- interpretable vs non-linear classifier comparison.

From an engineering perspective, the project is useful because thermal behaviour depends on dynamic interactions among CPU activity, cluster frequencies, power draw, system load, memory pressure, and other operating conditions. Temperature itself is a lagging outcome of underlying workload conditions, so predictive modelling is more useful than raw threshold alerts.

This makes the project richer than a simple temperature regression task. The pipeline attempts to capture **causal precursors and temporal signatures** of thermal build-up, then use them for classification.

4. Project framing

The project can be understood as moving through five major layers:

1. **Data acquisition layer** — collect telemetry from Apple Silicon sessions.
2. **Data organization layer** — store sessions in a reproducible folder and registry structure.
3. **Data preparation layer** — merge sessions, handle gaps, missingness, and label creation.
4. **Feature engineering layer** — transform raw signals into predictive covariates with temporal memory.
5. **Modelling layer** — compare classifiers and imbalance strategies.

At the moment, the project is largely complete through layer 4, and is about to enter layer 5.

5. What data we are collecting

The telemetry logger is designed specifically for Apple Silicon macOS systems and records structured performance and thermal information using **powermetrics**, **psutil**, and related system utilities. The logger writes one CSV per run and maintains a session registry for metadata tracking.

The output schema includes variables such as:

- timestamp,
- thermal pressure level and encoded thermal pressure code,
- CPU die temperature,
- GPU die temperature when available,
- CPU total activity,
- E-cluster and P-cluster activity,
- E-cluster and P-cluster frequencies,
- GPU activity and frequency,
- CPU, GPU, ANE, and combined power,
- CPU percent from **psutil**,
- load averages,
- RAM usage,
- memory pressure,
- swap usage,

- compressed memory,

This is already a strong telemetry base because it combines direct thermal indicators with indirect workload and resource-pressure signals.

6. Sessionized data collection design

A very important design choice in the project is that data are collected as **sessions**, not as one undifferentiated continuous log. Each run receives a machine-aware filename and registry entry, so the project preserves session boundaries and metadata such as machine ID, date, run number, start time, end time, duration, row count, interval, logger version, and notes.

This matters for several reasons:

- sessions can correspond to different workload regimes,
- session boundaries help avoid mixing heterogeneous conditions blindly,
- later train/validation splitting can be done by session rather than random row,
- registry metadata improves traceability and reproducibility.

The logger’s folder and naming design therefore contributes directly to later ML validity, not just convenience.

7. How the raw data are structured

The logger saves raw session files into a structured directory tree and also updates a session registry. The current design uses session-level storage under a machine/date hierarchy, with a separate registry for meta-data.

Conceptually, the raw data layer contains:

- **per-session telemetry CSVs** in `data/raw/`,
- **metadata registry** in `data/interim/sessionregistry.csv`,
- multiple sessions captured under different operating conditions.

The notebook logs show that several sessions were successfully merged, including recent sessions such as `macm520260625001`, `macm520260626001`, `macm520260626002`, `macm520260626003`, and `macm520260627001`.

This is important because it shows the project already moved beyond toy single-run experimentation and into multi-session analysis.

8. Role of the stress runner

The project also includes an intermittent stress runner, which is a very useful component conceptually because purely passive data collection may not generate enough thermal transitions or diverse workload states. The stress runner helps create **controlled workload variation** so that the dataset contains richer signal transitions and more informative pre-event patterns.

In other words, without some deliberate stressing, the model may mostly see benign or stable regimes, which would weaken both the positive class and the variability needed to learn predictive structure. The stress runner therefore supports dataset enrichment.

9. Data loading stage

After data collection, the first pipeline stage is to load and merge all available session CSVs into a unified dataframe for analysis. The notebook shows that the session loader successfully merged multiple sessions into one dataset, producing thousands of rows.

This stage is more than file I/O. It establishes:

- a consistent schema across sessions,
- ordering by time,
- a merged analysis table,
- session identity retained as a column.

This is the base table on which all later processing depends.

10. Data quality stage

Raw telemetry is rarely analysis-ready. The project includes a dedicated data quality layer for trimming sessions and handling temporal gaps, which is very important in time-series work because irregularities can corrupt both descriptive analysis and label construction.

The notebook logs explicitly show that:

- the first and last rows of sessions were trimmed,
- temporal gaps larger than the expected interval were detected,
- gap-related handling was applied before downstream stages.

This step matters because if timing is inconsistent, then a “1-minute horizon” is no longer well defined. Gap handling therefore protects the meaning of the prediction target.

11. Missing-value treatment

The next major stage handles missingness. The notebook logs show that missingness in at least some columns, such as CPU die temperature, was detected and then handled according to pipeline settings.

From a statistical perspective, missing-value treatment is necessary for two reasons:

1. most classifiers and scalers do not accept arbitrary missing entries,
2. telemetry missingness may itself reflect measurement limitations rather than true system state.

The preprocessing code indicates that the project uses a defined missingness threshold and treatment logic rather than silently dropping everything, which is the right design for reproducible ML preparation.

12. Label construction: the central learning formulation

This is one of the most important conceptual parts of the project.

The project creates a binary target Y by looking ahead in time and determining whether the target thermal condition will be reached soon. The notebook logs show a configuration using a horizon of 1.0 minute and 12 future steps, followed by dropping tail rows where the look-ahead window is incomplete.

This means the target is built approximately as follows:

- choose a threshold temperature, currently around 75.0 C in the notebook,
- for each time index, inspect the next fixed number of future steps,
- assign $Y = 1$ if the threshold will be crossed within that future window,
- assign $Y = 0$ otherwise,
- drop final rows that do not have a complete future window.

Formally, if temperature is denoted by T_t , threshold by τ , and the look-ahead horizon is H steps, then the target is conceptually:

$$Y_t = \mathbb{I} \left(\max_{h \in \{1, \dots, H\}} T_{t+h} \geq \tau \right)$$

This formulation is elegant because it translates a continuous physical process into a supervised classification target that has clear operational meaning.

13. What the current class distribution tells us

The latest notebook state shows a labelled dataset with 4,725 rows after trimming and look-ahead handling, and a class distribution of roughly 83.05% negatives and 16.95% positives.

This has several implications:

- the problem is **imbalanced**, though not catastrophically so,
- accuracy alone would be misleading,
- weighting or resampling should be tested,
- precision/recall trade-offs will matter more than overall error rate.

This is exactly why the next project phase is class imbalance treatment before full classifier comparison.

14. Exploratory data analysis

The project includes a separate EDA module and notebook-based descriptive analysis. The notebook summary shows descriptive statistics, target distributions, KDE plotting, timeline-style plots, regime/throttling plots, autocorrelation inspection, and baseline mutual-information style feature relevance analysis.

This stage is important because it answers questions such as:

- Are the core variables numerically reasonable?
- Are there outliers or heavy tails?
- How volatile are the thermal and power signals?
- Which variables appear most associated with the target?
- Is the target distribution plausible?
- How strong is temporal dependence?

That EDA work is not cosmetic. It directly informs threshold choices, feature design, and later model selection.

15. Signals that appear important already

The notebook output on baseline target signal importance suggests that variables such as combined power, CPU power, P-cluster activity, P-cluster frequency, total CPU activity, compressed memory, and some load measures already show meaningful predictive association with the target.

That result is intuitively sensible. Thermal build-up is often preceded by increases in power draw, sustained high-performance-cluster activity, and elevated workload pressure, so these variables are plausible precursors rather than arbitrary correlations.

This gives confidence that the dataset contains useful predictive signal.

16. Feature engineering philosophy

Raw telemetry rows are often too myopic for prediction because a single instant does not describe recent trajectory. The project therefore adds engineered features that summarize recent history. This is the key move that converts the problem from “instantaneous state classification” into “short-term dynamics prediction”.

The feature engineering design includes:

- lagged values,
- rolling means,
- rolling standard deviations,
- rolling maxima and minima,
- rolling ranges,
- first and multi-step differences,
- acceleration-like terms,
- domain-inspired proxy features.

This is exactly the right strategy for tabular ML on time-series-derived data.

17. What features were engineered

The notebook column output shows a large engineered feature set that includes examples such as:

- `cpudietempc_lag1, lag2, lag3, lag4, lag5, lag10,`
- `cpupowermw_lag1...lag10,`
- `combinedpowermw_lag1...lag10,`
- `cpupclusteractivepct_lag1...lag10,`
- `cpupclusterfreqmhz_lag1...lag10,`
- rolling statistics over 30s, 1m, 5m, and some longer windows,
- first differences and 6-step differences,
- acceleration features,
- proxy features like `thermaldistressproxy` and `pclusterpowerpressure`.

These features allow the model to infer whether a signal is rising, stable, noisy, or recently extreme, which is crucial for near-future prediction.

18. Boundary-row removal after feature construction

Feature engineering with lags and rolling windows inevitably creates boundary rows whose history is incomplete. The notebook logs show that a set of boundary rows was dropped after feature engineering.

This is the correct decision. Keeping these rows would inject invalid or partially defined feature values and compromise training quality.

19. What has been achieved so far

At this checkpoint, the project has already accomplished a substantial amount:

- built a custom telemetry logger for Apple Silicon,
- built a session metadata registry,
- created a stress-generation utility,
- defined a structured raw/interim data layout,
- merged multiple telemetry sessions,
- handled trimming, gaps, and missingness,
- defined a forward-looking binary label,
- performed descriptive analysis and target verification,
- engineered a large temporal feature set,
- produced an ML-ready dataset just before classifier training.

So the project is not in an early “idea” stage. It is already at the important transition between **data/feature pipeline completion** and **predictive modelling**.

20. What tools and libraries were utilised

The project uses a mix of Python ecosystem tools and macOS system utilities.

Systems / collection side

- `powermetrics` for Apple Silicon telemetry,
- `psutil` for CPU, memory, battery, and related system metrics,
- standard Python libraries such as `argparse`, `csv`, `logging`, `subprocess`, `pathlib`, `datetime`, and `plistlib`.

Data science / ML side

From the notebook and module structure, the project clearly uses the standard tabular data stack, including pandas-style dataframe workflows, plotting/EDA tooling, and scikit-learn-compatible ML preparation.

The companion repository files prepared for GitHub also reflect expected dependencies such as pandas, numpy, matplotlib, seaborn, scikit-learn, scipy, psutil, Jupyter, and imbalanced-learn.

21. Why the project is modularised

A very good design decision in the project is that functionality is split across modules such as data loading, quality checks, transformation, labels, EDA, features, and broader preparation helpers.

This is preferable to placing everything directly in one large notebook because modularisation improves:

- readability,
- reuse,
- debugging,
- testing,
- teammate onboarding,
- maintainability.

The notebook then acts as the orchestration and experimentation layer rather than a monolithic code dump.

22. Statistical learning interpretation

From an SML perspective, the final learning problem is a binary classification problem with engineered covariates derived from temporally ordered observations. If features are denoted by $x_t \in \mathbb{R}^p$, then the model will learn a mapping:

$$f : \mathbb{R}^p \rightarrow [0, 1]$$

where $f(x_t)$ estimates the probability that thermal distress will occur within the next horizon. The hard prediction is then:

$$\hat{Y}_t = \mathbb{I}(f(x_t) \geq \gamma)$$

for some decision threshold γ .

This framing immediately explains why threshold selection and class imbalance treatment matter. The default probability cutoff of 0.5 may not be optimal for an early-warning task.

23. Why class imbalance is the immediate next step

Because only about 16.95% of the rows are positive in the current notebook state, a naive classifier could achieve superficially decent accuracy simply by favouring the negative class.

That would be unacceptable for the real objective, because missing upcoming thermal events is costlier than merely labelling too many benign states as risky. Therefore, the project's next stage should compare imbalance-aware methods such as:

- class-weighted losses,
- random oversampling / undersampling,
- SMOTE-type resampling,
- threshold moving after probabilistic prediction.

The important methodological caution is that any such balancing must be applied only within the training partition or training folds, never globally before splitting, otherwise leakage occurs.

24. Planned model family comparison

The intended model sequence is well:

Logistic Regression

Random Forest

Gradient Boosting

Neural Network

This progression is methodologically sound because it starts with interpretable baselines and then increases model complexity.

25. Recommended evaluation logic

For this task, evaluation should not revolve around accuracy. Because the dataset is imbalanced and the task is early warning, the most important metrics are those that focus on the positive class and the ranking quality of probabilistic predictions.

Recommended evaluation includes:

- precision,
- recall,
- F1-score,
- confusion matrix,
- PR-AUC,
- ROC-AUC,
- calibration quality,
- possibly event-level or lead-time usefulness analysis.

A model with slightly lower overall accuracy but materially better positive-class recall and acceptable precision may be far more valuable here.

26. Leakage risks and methodological care

This project has multiple possible leakage points, so it is good that the team is pausing now to document the pipeline properly before diving into model training.

Leakage risks include:

- fitting imputers/scalers on the full dataset,
- oversampling before splitting,
- using future-derived information inside features,
- random row-level splitting that leaks session-specific structure,
- forgetting that temporal order matters.

A careful training design should therefore prefer session-aware or time-aware splits, and ensure that every preprocessing object is fitted only on the training data.

27. End-to-end pipeline summary

A concise way to describe the whole workflow is:

1. Collect Apple Silicon telemetry in sessionized CSV form.
2. Register each session with metadata.
3. Merge multiple sessions into one dataset.
4. Clean edge effects and temporal gaps.
5. Treat missing values appropriately.
6. Construct a forward-looking thermal-event label.
7. Validate target distribution and inspect signals with EDA.
8. Engineer temporal features from recent history.
9. Remove invalid boundary rows.
10. Handle imbalance and train classifiers.
11. Evaluate and compare models.
12. Later package the best pipeline for reuse.

This is the backbone of the entire project.

28. Current project status in one paragraph

The project currently stands at a strong intermediate milestone: the telemetry collection framework is working, multiple sessions have been merged, gaps and missingness have been handled, a forward-looking binary label has been created, extensive EDA has been performed, and a large feature set has been engineered to create an ML-ready table. The immediate next steps are to address class imbalance carefully, train and compare Logistic Regression, Random Forest, Gradient Boosting, and Neural Network models, evaluate them with positive-class-sensitive metrics, and then refine the best-performing approach into a reusable predictive pipeline